



Project Report on Mini Compiler

Submitted to:

Khalid Bin Selim

Lecturer

Department of Computer Science & Engineering

Metropolitan University, Sylhet

Submitted by:

Debopriyo Das Rahul (231-115-126)

Amit Banik (231-115-144)

Jasmin Amin Tarafder (222-115-016)

Section: D

Batch: 58

Department of Computer Science and Engineering

Metropolitan University, Sylhet

Date of submission: 2nd August, 2026

Table of Contents

1. Introduction.....	2
2. Objectives.....	2
3. Language Specification.....	2
3.1 Formal Context-Free Grammar.....	3
3.2 Precedence and Associativity.....	4
4. Compiler Architecture.....	4
4.1 Module-to-file map.....	5
4.2 Build order.....	6
5. Lexer Design.....	6
6. Parser Design.....	7
7. Abstract Syntax Tree.....	7
8. Symbol Table.....	8
9. Semantic Analysis.....	9
10. Intermediate Code.....	10
11. Testing.....	11
11.1 Successful compilation.....	11
11.2 Lexical error.....	11
11.3 Syntax error and recovery.....	11
11.4 Semantic errors — one test per rule (Section 4.5).....	12
12. Challenges.....	12
13. Testing Summary.....	13
14. Conclusion.....	13
15. References.....	14

1. Introduction

Compilers translate programs written in a high-level source language into a form that a machine, or a later stage of a toolchain, can execute. Every mainstream compiler is organised as a pipeline of phases — lexical analysis, syntax analysis, semantic analysis, and code generation — each of which was studied in isolation across Labs 1 through 4 of this course. This project asks us to stop treating these phases as separate exercises and instead wire them into a single, working compiler front end.

We were given a fixed, instructor-specified language (Project Manual, Section 5): a small imperative language with `int`, `float`, and `bool` types, arithmetic/relational/logical expressions, `if/if-else`, `while`, `print`, and nested block scoping. Every group implements the same language so that design decisions and code quality can be compared fairly, rather than comparing projects of differing difficulty.

This report documents the compiler we built: Group 03's implementation, covering the full pipeline from raw source text down to Three-Address Code (TAC).

2. Objectives

The project sets out to demonstrate, through working code, that we can:

- Build a Flex-based lexical analyser that tokenises the language, discards whitespace and comments, and reports lexical errors with line numbers.
- Build a Bison-based syntax analyser that implements a complete, unambiguous CFG for the language, with panic-mode error recovery so a single syntax error does not abort the whole parse.
- Construct an Abstract Syntax Tree (AST) as the shared data structure passed between the parser, the semantic analyser, and the code generator.
- Implement a symbol table that supports nested scopes and rejects redeclaration within the same scope.
- Implement a semantic analyser that enforces type rules the grammar itself cannot express (undeclared-variable use, redeclaration, scope violation, type mismatch, invalid assignment, invalid expressions).
- Generate Three-Address Code for expressions, control flow (`if`, `if-else`, `while`), and `print` statements.

- Demonstrate all of the above on both valid and deliberately invalid test programs, showing that the compiler fails gracefully and informatively.

3. Language Specification

The implemented language is exactly the one defined in Project Manual, Section 5.

Data types: int, float, bool.

Statements: variable declaration, assignment, if, if-else, while, print, nested { ... } blocks.

Operators:

Category	Operators
Arithmetic	+ - * / %
Relational	< > <= >= == !=
Logical	&& \ \ !

Lexical elements: identifiers ([a-zA-Z_][a-zA-Z_0-9]*), integer literals, floating-point literals, boolean literals (true/false), { } () ;, // single-line comments, and /* */ block comments.

3.1 Formal Context-Free Grammar

The grammar below is transcribed directly from src/parser/parser.y.

```

program      ::= stmt_list

stmt_list    ::= stmt_list stmt
               | /* empty */

stmt         ::= decl
               | assign
               | if_stmt
               | while_stmt
               | print_stmt
               | block
               | error ';'                /* panic-mode recovery */

block        ::= '{' stmt_list '}'

type         ::= 'int' | 'float' | 'bool'

decl         ::= type Identifier ';'

```

```

assign      ::= Identifier '=' expr ';'

if_stmt     ::= 'if' '(' expr ')' block
              | 'if' '(' expr ')' block 'else' block

while_stmt  ::= 'while' '(' expr ')' block

print_stmt  ::= 'print' expr ';'

expr        ::= expr '+' expr | expr '-' expr | expr '*' expr
              | expr '/' expr | expr '%' expr
              | expr '<' expr | expr '>' expr | expr '<=' expr | expr '>='
expr
              | expr '==' expr | expr '!=' expr
              | expr '&&' expr | expr '||' expr
              | '!' expr
              | '-' expr %prec UMINUS
              | '(' expr ')'
              | Identifier | IntLit | FloatLit | 'true' | 'false'

```

3.2 Precedence and Associativity

Declared lowest to highest, matching the %left/%right declarations in parser.y:

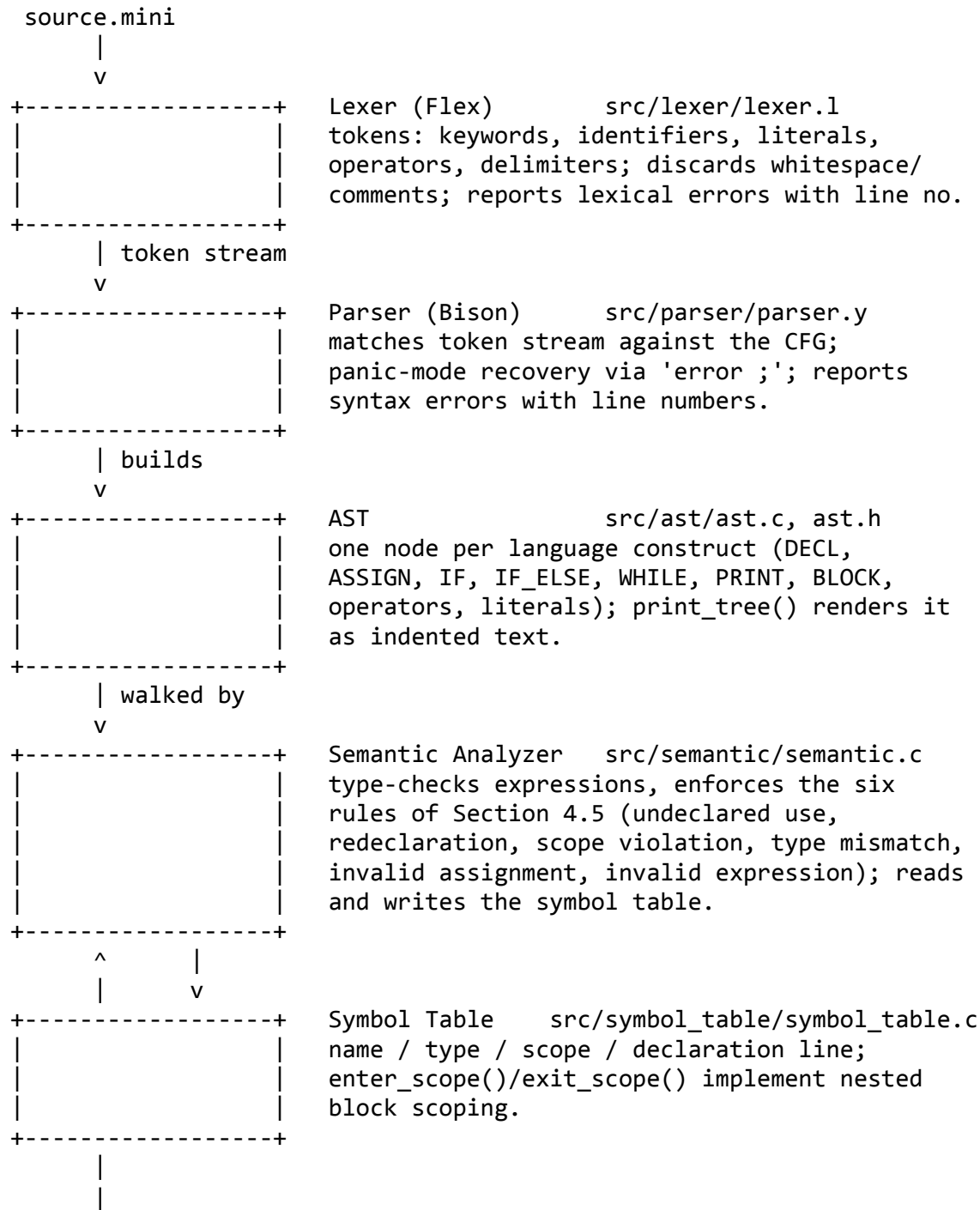
Level (low → high)	Operators	Associativity
1	\ \	left
2	&&	left
3	== !=	left
4	< > <= >=	left
5	+ -	left
6	* / %	left
7	!, unary -	right

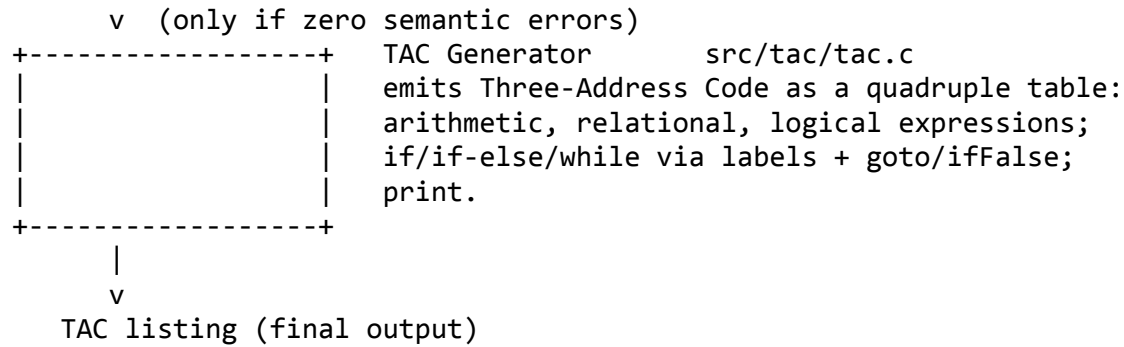
Unary minus is disambiguated from binary minus using the standard %prec UMINUS trick, since Bison cannot otherwise tell which precedence a '-' expr rule should take.

The only conflict-relevant grammar addition beyond ordinary expression precedence is the error ';' production, added purely for panic-mode syntax-error recovery; it does not interact with expression precedence, and bison -d reports zero unresolved shift/reduce or reduce/reduce conflicts for this grammar.

4. Compiler Architecture

The compiler is a straight, top-to-bottom pipeline with no backtracking between phases — the semantic analyser never triggers a re-parse, and TAC generation is skipped entirely whenever the semantic analyser reports any errors, rather than generating TAC from an invalid program.





4.1 Module-to-file map

Phase	Files
Lexer	src/lexer/lexer.l (generates lex.yy.c)
Parser	src/parser/parser.y (generates parser.tab.c / .h)
AST	src/ast/ast.c, src/ast/ast.h
Symbol Table	src/symbol_table/symbol_table.c, .h
Semantic Analysis	src/semantic/semantic.c, .h
TAC Generation	src/tac/tac.c, .h
Driver (main)	src/parser/parser.y (epilogue)

4.2 Build order

Makefile enforces the dependency order required by Flex/Bison integration: Bison runs first on parser.y to produce parser.tab.h (the token definitions), because lexer.l #includes that header; only then does Flex run on lexer.l; GCC then compiles and links every module together into bin/compiler.

1. bison -d src/parser/parser.y -> parser.tab.c, parser.tab.h
2. flex src/lexer/lexer.l -> lex.yy.c
3. gcc ast.c symbol_table.c semantic.c tac.c parser.tab.c lex.yy.c -> bin/compiler

5. Lexer Design

src/lexer/lexer.l is a single-pass Flex scanner. Token definitions used throughout:

```

DIGIT      [0-9]
LETTER     [a-zA-Z_]
IDENTIFIER  {LETTER}{(LETTER|DIGIT)}*
```

```
INT_LIT      {DIGIT}+
FLOAT_LIT    {DIGIT}+"."{DIGIT}+
```

Key design decisions:

- **Keywords before identifiers.** `int`, `float`, `bool`, `if`, `else`, `while`, `print`, `true`, and `false` are matched as literal-string rules that appear before the generic `{IDENTIFIER}` rule, so Flex's longest-match/first-rule ordering resolves keyword-vs-identifier ambiguity without a separate keyword table.
- **FLOAT_LIT before INT_LIT.** Although Flex's longest-match rule alone is sufficient to prefer `3.14` over `3`, the float rule is still placed first for readability and to make the ordering intent explicit, following the Lab 1 tip.
- **%option yylineno** is enabled so every token carries its source line number automatically, which the parser and semantic analyser reuse for error messages without maintaining a separate line counter.
- **Two comment styles.** `"/"/.*` discards single-line comments; a regex for block comments `("/*"([^\*]|\\*+[^*/])*\*+"")` discards `/* ... */` comments, including ones spanning content with embedded `*` characters that are not immediately followed by `/`.
- **Multi-character operators before their single-character prefixes.** `==`, `!=`, `<=`, and `>=` are declared before `<` and `>` so the scanner does not greedily match `<` and then fail on a dangling `=`.
- **Lexical error reporting.** The catch-all rule `.` reports `Lexical Error: invalid token '<char>' at line <n>` to `stderr` and continues scanning, rather than aborting — this lets a single pass surface every invalid character in the input, not just the first.

6. Parser Design

`src/parser/parser.y` implements the CFG of Section 3.1 directly as Bison rules. A `%union` gives `yylval` (and `$$/$n`) four possible payload types — `int`, `double`, `char *`, and `struct ASTNode *` — since different tokens and non-terminals carry different kinds of semantic value (an `IDENTIFIER` carries a string, an `INT_LIT` carries an `int`, every grammar non-terminal that represents part of the program carries an `ASTNode *`).

Every grammar rule's action allocates one AST node (via `make_node()`) and attaches the actions of its right-hand-side symbols as children (via `add_child()`) — the same $\$/$ -driven, bottom-up tree-construction pattern used for the parse tree in Lab 3, except that no node is created for pure punctuation (`( ) { } ;`), which is what turns a Lab-3-style *parse* tree into an *abstract* syntax tree.

Error recovery. The rule `stmt: error SEMI { ... yyerrok; }` implements Bison's panic-mode recovery: on a syntax error, Bison discards tokens until it can shift a `;`, builds a placeholder `ERROR` AST node for that statement, calls `yyerrok` to reset the parser's error

state, and resumes parsing the next statement — so one malformed statement is reported without preventing the parser from checking the rest of the file in the same pass.

Line numbers. Every `make_node()` call in the grammar actions passes `yylineno`, so every AST node — not just leaves — is annotated with the source line it came from, which the semantic analyser later reports in its error messages.

7. Abstract Syntax Tree

`src/ast/ast.h` defines a single generic node type (the same design approach as the parse-tree Node struct of Lab 3), rather than one C struct per construct:

```
typedef struct ASTNode {
    char label[32];    /* e.g. "IF", "WHILE", "ASSIGN", "DECL", "+", or a
name/literal */
    int line;
    NodeKind kind;     /* NK_OTHER / NK_IDENT / NK_INT_LIT / NK_FLOAT_LIT /
NK_BOOL_LIT */
    int ival; double fval;
    int num_children;
    struct ASTNode *children[MAX_CHILDREN];
} ASTNode;
```

The `label` field distinguishes node types textually (“IF”, “WHILE”, “ASSIGN”, “DECL”, an operator such as “+”, or an identifier/literal’s own text), which keeps the grammar actions in `parser.y` short and uniform. The `NodeKind` enum was added specifically because the label alone is ambiguous for leaves — a float literal such as `100.0` prints as “100” under `%g` formatting, which is textually identical to the int literal `100` — so the semantic analyser needs `kind` to tell an identifier, an int literal, a float literal, and a bool literal apart reliably, rather than re-parsing the label string.

`print_tree()` renders the AST as indentation-based text, bracketing internal (non-leaf) nodes as `[LABEL]` and leaving leaves unbracketed, e.g. for `c = a + b * 2;`:

```
[ASSIGN]
  c
  [+ ]
    a
    [*]
      b
      2
```

8. Symbol Table

src/symbol_table/symbol_table.c stores each declared identifier's name, type, declaring scope level, and declaration line, exactly as required by Project Manual Section 4.4:

```
struct symbol {  
    char name[32];  
    char type[12];  
    int scope;  
    int line;  
    int active;  
};
```

Nested scoping. `enter_scope()` is called when a block's `{` is opened and `exit_scope()` when the matching `}` closes. Symbols are inserted only into the *current* scope; `lookup()` searches outward from the current scope through enclosing scopes, so the nearest (innermost) declaration wins — this correctly implements shadowing, where an inner block's `int x;` temporarily hides an outer `x` for the remainder of that block only.

The active flag. Rather than deleting a symbol's entry when its scope closes, `exit_scope()` marks it `active = 0` and `lookup()` simply skips inactive entries. This design choice means `print_symtab()` can show the *complete history* of every symbol ever declared in the program (labelled *active* or *exited*), which is more useful for demonstration and viva purposes than a table that has already forgotten variables from closed blocks.

Redeclaration. `insert()` checks the current scope only (not enclosing scopes) and returns failure if the name is already present there, which is exactly the semantic-analyser hook used to detect Section 4.5's redeclaration rule — declaring `int x;` twice in the *same* block is an error, but re-declaring `x` in a *nested* block is legal shadowing.

9. Semantic Analysis

src/semantic/semantic.c walks the AST produced by the parser and enforces every rule listed in Project Manual, Section 4.5. `analyze_expr()` is a recursive, bottom-up type checker: it returns the inferred type of an expression subtree ("int", "float", "bool", or the sentinel "error"), and every statement-level check (assignment, condition, print) calls it on its sub-expressions.

Rule	How it is detected
Undeclared variable use	<code>lookup()</code> on an identifier returns -1
Redeclaration	<code>insert()</code> into the current scope fails because the name already exists there

Rule	How it is detected
Scope violation	A variable used after its declaring block has closed is, by construction, no longer visible to <code>lookup()</code> and is reported identically to an undeclared-variable error — the symbol table's active flag is what makes a scope-exited variable invisible again
Type mismatch	e.g. arithmetic and relational operators require both operands to be <code>int/float</code>
Invalid assignment	The right-hand side's inferred type is incompatible with the declared type of the left-hand side variable
Invalid expressions	e.g. <code>&&</code> and <code>\ \ </code> require <code>bool</code> operands; applying them to <code>int</code> operands is rejected

Error cascading is suppressed. When a subtree already produced an error, `analyze_expr()` returns the sentinel type "error" instead of a real type, and every calling rule short-circuits on "error" operands without emitting a second complaint about the same broken subtree. This mirrors real compilers such as GCC, which avoid burying the first, actionable error under a cascade of consequential ones.

Two concrete examples from the equality-operator rule and the arithmetic rule illustrate the type-widening policy: `int == float` is allowed (both are numeric, so they may be compared) and produces `bool`; `int + float` is allowed and widens to `float`, matching ordinary numeric-promotion rules. Every reported error includes the line number recorded on the offending AST node, satisfying the human-readable, line-numbered error message requirement of Section 4.5.

10. Intermediate Code

`src/tac/tac.c` walks a *semantically valid* AST (TAC generation is only invoked once `analyze()` returns zero errors) and emits Three-Address Code as a quadruple table (`op`, `arg1`, `arg2`, `result`), following the same `newTemp()/emit()` pattern introduced in Lab 4.

Expressions. Each binary operator node emits one quadruple and returns a fresh temporary standing for its result, e.g. for `c = a + b * 2`:

```
a = 5
b = 10
t1 = b * 2
```

```
t2 = a + t1
c = t2
```

Control flow. `if/if-else` and `while` are lowered using generated labels (`L1`, `L2`, ...) and `ifFalse ... goto / goto` instructions, exactly the patterns given in Lab 4, Section 2.5. For example, `while (x > 0) { y = y + x; x = x - 1; }` lowers to:

```
L1:
t1 = x > 0
ifFalse t1 goto L2
t2 = y + x
y = t2
t3 = x - 1
x = t3
goto L1
L2:
```

print statements evaluate their expression into a temporary (or reuse an existing variable/temporary directly) and emit a `print <value>` instruction.

TAC generation is deliberately skipped whenever `main()` (in `parser.y`) sees a non-zero semantic error count, so a program that fails semantic analysis produces no TAC at all — this is an intentional design decision (documented in `docs/architecture.md`), not a missing feature.

11. Testing

All test programs live under `tests/`, each paired with its expected output in a matching `.out` file, and are run automatically by `run_tests.sh`. The suite covers every category required by Project Manual Section 15.

11.1 Successful compilation

`examples/01_variables_and_arithmetic.mini` compiles cleanly end-to-end:

```
int a; int b; int c;
a = 5; b = 10; c = a + b * 2;
print c;
```

produces the AST, an empty (all-active) symbol table, and the TAC:

```
a = 5
b = 10
t1 = b * 2
t2 = a + t1
c = t2
print c
```

examples/03_nested_scope.mini was used specifically to demonstrate scoping: an inner `int x;` inside an `if` block shadows the outer `x` only within that block, and the symbol table lists both entries at different scope levels, one active and one exited at the end of the run.

11.2 Lexical error

tests/04_lexical_error_invalid_char.mini (`x = 5 @ 3;`) produces:

Lexical Error: invalid token '@' at line 2

and — since `@` is also not valid at that point in the grammar — a follow-on syntax error, after which the AST shows a placeholder `ERROR` node for that statement and semantic analysis is skipped for the file.

11.3 Syntax error and recovery

tests/06_syntax_error_if_no_braces.mini (an `if` whose body is a single bare statement, with no `{ }`) produces `Syntax Error: syntax error at line 4`. The parser recovers via the `error ;` production, so the earlier `int x;` and `x = 1;` statements still appear correctly in the AST.

11.4 Semantic errors — one test per rule

Rule	Test file	Reported message
Undeclared variable	tests/07_semantic_error_undeclared.mini	Semantic Error (line 2): undeclared variable 'y'
Redeclaration	tests/08_semantic_error_redeclaration.mini	Semantic Error (line 2): redeclaration of variable 'x'
Scope violation	tests/09_semantic_error_scope_violation.mini	Semantic Error (line 5): undeclared variable 'x'
Type mismatch	tests/10_semantic_error_type_mismatch.mini (<code>bool b; b = 5 + 3.2;</code>)	Semantic Error (line 2): invalid assignment: cannot assign float to variable 'b' of type bool
Invalid assignment	tests/11_semantic_error_invalid_assignment.mini (assigning a bool to an int)	Semantic Error (line 4): invalid assignment: cannot assign bool to variable 'x' of type int

Rule	Test file	Reported message
Invalid expression	tests/12_semantic_error_invalid_expression.mini (if (x && 3))	Semantic Error (line 3): operator '&&' requires boolean operands, got int and int

Note that a **scope violation** and an **undeclared variable** are surfaced through the same underlying mechanism and the same message text — a variable used outside the block that declared it is, by construction, invisible to `lookup()` once that scope has exited, so from the semantic analyser’s point of view the two cases are indistinguishable. This is discussed further in Section 12 (Challenges).

12. Challenges

- **Distinguishing scope violation from undeclared-variable errors.** Because the symbol table’s `lookup()` simply skips `active == 0` entries, a variable used after its scope has closed produces exactly the same `-1` result — and hence the same error message — as a variable that was never declared at all. We considered keeping exited entries visible to `lookup()` purely so the message could say “declared but out of scope” instead of “undeclared”, but decided the simpler, uniform behaviour was preferable: real compilers such as GCC also frequently just say “not declared in this scope” for both cases, and our symbol table’s exited status is still fully visible in `print_symtab()` for debugging.
- **Float literals printing as int-like text.** `%g` formatting of a float such as `100.0` prints as “100”, textually identical to the int literal `100`. This broke early attempts at inferring a leaf’s type purely from its `label` string, and is the reason `NodeKind` was added to `ASTNode` — the parser now tags each literal/identifier leaf with its true kind at construction time instead of the semantic analyser trying to re-derive it later.
- **Ordering bison before flex in the build.** `lexer.l` needs the token codes Bison generates in `parser.tab.h`, but `parser.y` also needs the AST/semantic/TAC headers to compile its actions. Getting the Makefile’s dependency graph right (Bison first, then Flex, then GCC over everything) took a few iterations before settling on the two explicit pattern rules shown in Section 4.2.
- **Not letting a broken subtree cascade into many bogus errors.** An early version of the semantic analyser reported a fresh type-mismatch error for every ancestor of a single bad expression (e.g. an undeclared variable used inside three nested arithmetic operators produced four error lines for one bug). Introducing the “error” sentinel return type, and having every check short-circuit on it, brought this down to exactly one reported error per genuine problem.
- **Suppressing TAC output for invalid programs without special-casing every phase.** Rather than threading an error flag through the TAC generator itself, `main()` in `parser.y` simply never calls `generate_tac()/print_tac()` when `analyze()`

returns a nonzero error count — keeping the “don’t generate code for broken programs” policy in one place instead of scattered checks.

13. Testing Summary

`run_tests.sh` builds the project and runs the compiler against every `.mini` file in `tests/`, covering: one non-trivial valid program compiling cleanly through to TAC (`tests/01–03`), a lexical error (`tests/04`), syntax errors including multi-error recovery (`tests/05, 06, 13`), and one test for every semantic rule in Section 4.5 (`tests/07–12`). Each `.mini` file is paired with a corresponding `.out` file recording the expected output, so regressions in any phase are caught by comparing fresh output against the recorded baseline.

14. Conclusion

Building this compiler made the interfaces between compiler phases concrete in a way that studying them separately in Labs 1–4 did not. In particular: how much information a symbol table has to carry forward (not just “is this declared” but “in which scope, and is that scope still open”) to make semantic analysis correct; how a single generic AST node type, annotated with a `kind` field, is enough to represent an entire language without one C struct per construct; and how deliberately *not* cascading errors, and deliberately *not* generating code for an invalid program, are both design decisions that have to be made on purpose rather than falling out “for free” from a naive implementation. The project also reinforced that a compiler’s job is only half done by accepting valid programs — a compiler that rejects invalid ones with a clear, line-numbered explanation is, as the Project Manual states, just as valuable academically as one that only handles the happy path.

15. References

- Compiler Construction Lab, Project Manual and Lab 1–4 Manuals, Metropolitan University, Department of CSE.
- GNU Bison Manual — <https://www.gnu.org/software/bison/manual/>
- Flex Manual — <https://westes.github.io/flex/manual/>